SRS — Đặc tả yêu cầu, Brain, chu kỳ 01

1. Về tài liệu này
   - 1.1. Nó là gì, và không là gì
   - 1.2. Khuôn một yêu cầu, dùng lại cho chu kỳ sau
   - 1.3. Quy ước mã và mức ưu tiên
   - 1.4. Từ khoá mức bắt buộc
2. Bối cảnh
   - 2.1. Brain làm gì, và ranh giới
   - 2.2. Lộ trình nói việc kế tiếp là gì
   - 2.3. Ràng buộc phần cứng, và vì sao L4 là sàn
   - 2.4. Hiện trạng đo được, mốc `645f4a1`
   - 2.5. Ràng buộc bất biến
3. Kiến trúc đích của chu kỳ
   - 3.1. Hiện trạng đường sinh text
   - 3.2. Đích
   - 3.3. Ba cái bẫy đã xác định trong đường server
4. Yêu cầu chức năng
   - BR-01. `/tools/vat` phải từ chối trường lạ thay vì tính bằng 0
   - BR-02. Thêm đường chat cho `VLLMServerClient`
   - BR-03. Cho `engine.py` rẽ nhánh theo backend, và chứng minh đường HTTP thi hành
   - BR-04. Dựng vLLM server Qwen3-8B trên Colab L4 và đo ngân sách ngữ cảnh
   - BR-05. Chạy lại phiên đo trên đường server, đóng việc treo số 1 và số 2
   - BR-06. Bộ đo RAG
   - BR-07. Mở rộng cổng lint của CI
5. Yêu cầu phi chức năng
6. Phân công và lịch
   - 6.1. Hai luồng việc, chia theo năng lực chứ không theo tên
   - 6.2. Đường găng
   - 6.3. Tuần 1: Thứ Hai 26/10 đến Thứ Sáu 30/10
   - 6.4. Tuần 2: Thứ Hai 02/11 đến Thứ Sáu 06/11
   - 6.5. Hai mốc kiểm
7. Cổng ra chu kỳ
8. Rủi ro, giả định, việc cần chốt
   - 8.1. Rủi ro
   - 8.2. Giả định
   - 8.3. Bốn quyết định của chủ dự án, chốt ngày 24/10/2026
   - 8.4. Còn để ngỏ
9. Ngoài phạm vi chu kỳ 01
10. Tham chiếu

SRS — Đặc tả yêu cầu, Brain, chu kỳ 01

Chu kỳ: Thứ Hai 26/10/2026 → Thứ Sáu 06/11/2026 (10 ngày làm việc) **Lập ngày:** 24/10/2026 · **Mốc mã nguồn:** `feat/workflow-format-and-deterministic-core` @ `645f4a1` **Phạm vi:** chỉ repo Brain (`PCBoiz/ANSER_AI`). Body có bản bàn giao riêng **Trạng thái:** bản thảo, chờ chủ dự án chốt §8.3

> **Đính chính bản trước.** Bản SRS đầu (24/10, sáng) đặt trọng tâm vào việc triển khai lớp kế toán trên máy CPU, và đẩy **S2** cùng **phiên đo Colab** vào mục ngoài phạm vi. Đó là sai: AGENTS.md §8 đặt S2 là việc 🔵 **kế tiếp**, và TIEN_TRINH_BRAIN.md §7 đặt phiên đo Colab là việc **số 1**. Bản này dựng lại quanh đúng hai thứ đó.
>
> Phần còn đúng của bản trước được giữ: khuyết tật `/tools/vat` (BR-01) và khoảng trống triển khai. Khoảng trống triển khai nay nằm ở §9 kèm lý do, thay vì chiếm chỗ của S2.

---

1. Về tài liệu này

1.1. Nó là gì, và không là gì

Đây là **đặc tả cho người viết mã**. Mỗi yêu cầu ở §4 và §5 có một tiêu chí nghiệm thu **chạy được bằng một lệnh**. Yêu cầu nào không kiểm được bằng lệnh thì nó chưa phải yêu cầu, nó là một mong muốn, và nó không được nằm trong tài liệu này.

Nó **chỉ nói về Brain**. Công việc của Body nằm ở bản bàn giao riêng của Body. Chỗ nào Brain phụ thuộc Body, tài liệu này nêu đúng thứ cần và để nguyên ở đó.

Nó không thay SO_TAY_NGUOI_MOI.md (dựng môi trường, hiểu mã) hay GIAO_VIEC_BRAIN.md (bảy việc dài hạn kèm bối cảnh). Tài liệu này cắt ra đúng phần làm trong hai tuần tới, gán người và hạn.

1.2. Khuôn một yêu cầu, dùng lại cho chu kỳ sau

Chu kỳ 02 viết theo đúng khuôn này. Sáu mục, không thêm không bớt:

```
### <MÃ>. <tên ngắn, động từ trước>
**Ưu tiên** P0|P1|P2 · **Người làm** <vai> · **Hạn** <thứ, ngày> · **Phụ thuộc** <mã hoặc "không

**Vì sao**        một đoạn. Nêu hậu quả cụ thể nếu không làm, tốt nhất là một
                  sự cố đã xảy ra. Không có hậu quả cụ thể thì xem lại yêu cầu.

**Yêu cầu**       dùng PHẢI / NÊN / CÓ THỂ. Mỗi câu một mệnh đề kiểm được.

**Nghiệm thu**    lệnh chạy được, kèm đầu ra mong đợi. Người khác chạy phải ra
                  cùng kết quả mà không cần hỏi gì thêm.

**Ngoài yêu cầu** nói rõ cái gì KHÔNG thuộc phạm vi, để tránh phình việc.
```

Mục **Ngoài yêu cầu** hay bị bỏ nhất và cứu nhiều thời gian nhất.

1.3. Quy ước mã và mức ưu tiên

Tiền tố	Nghĩa
`BR-`	Yêu cầu thuộc repo Brain
`NFR-`	Yêu cầu phi chức năng, áp cho mọi yêu cầu chức năng

Mức	Nghĩa
P0	Không xong thì cổng ra chu kỳ (§7) không mở
P1	Cần cho chu kỳ, trượt thì vẫn mở được cổng kèm ghi chú
P2	Làm nếu còn thời gian. Trượt thì đẩy sang chu kỳ 02, không cần giải trình

1.4. Từ khoá mức bắt buộc

PHẢI là điều kiện nghiệm thu. **NÊN** là khuyến nghị mạnh, bỏ qua thì ghi lý do vào PR. **CÓ THỂ** là tuỳ chọn.

---

2. Bối cảnh

2.1. Brain làm gì, và ranh giới

Brain là dịch vụ FastAPI cung cấp năng lực AI qua HTTP: lõi tính toán tất định (`/tools/*`), hỏi đáp có vòng agentic (`/chat`), đọc tài liệu (`/knowledge`), đọc ảnh hoá đơn (`/ocr`). Body gọi vào, n8n gọi vào.

Theo AGENTS.md §1, người làm Brain **không sửa repo Body**. Chu kỳ này không có yêu cầu nào cần Body đổi.

2.2. Lộ trình nói việc kế tiếp là gì

Hai tài liệu kiểm soát, cùng chỉ một hướng:

AGENTS.md §8 liệt kê tám hạng mục đã xong và một hạng mục kế tiếp:

> **S2** · Hạ tầng: **vLLM server riêng, Qwen3-8B, ctx 32k, Redis, Cloudflare Tunnel** · 🔵 **Kế tiếp**

TIEN_TRINH_BRAIN.md §7 đề nghị thứ tự:

#	Việc	Chặn bởi
1	**Phiên đo Colab**	một buổi GPU L4
2	**Bộ đo RAG**	không
3	Kế toán thật rà kết quả	tìm được người
4	Nối `redis` cho TASK_REGISTRY	không, chưa gấp
5	Nối `vllm-vision` qua HTTP	không

Việc 1 và 2 làm song song được, vì một cần GPU và một không.

Chu kỳ 01 nhận nửa đầu của S2 cộng việc 1 và 2. S2 lớn hơn một chu kỳ: Redis, vision qua HTTP và Cloudflare Tunnel sang chu kỳ 02, lý do ở §9.

2.3. Ràng buộc phần cứng, và vì sao L4 là sàn

Model text là `Qwen/Qwen3-8B` ở định dạng AWQ 4-bit. Không có Qwen3.5 ở đâu trong repo; rà toàn bộ mã, tài liệu và notebook cho ra 22 lần nhắc `Qwen3-8B` và không lần nào nhắc 3.5.

> ⚠️ **Bảng VRAM ở ARCHITECTURE.md §5.1 đã lỗi thời.** Nó ghi vision `Qwen2.5-VL-3B AWQ ~3,0 GB` và tổng `~15,0 GB`. Mã không làm thế: `engine.py` nạp vision bằng `torch_dtype=torch.bfloat16`, **không lượng tử hoá gì cả**. RAG_VLM_KE_HOACH.md §B.2.4 đã tính lại ngày 03/08/2026 và bảng đó mới là bảng đúng. Dùng bảng dưới, đừng dùng §5.1.

Ngân sách VRAM thật trên L4 22,5GB, theo RAG_VLM_KE_HOACH.md §B.2.4:

Thành phần	VRAM	Ghi chú
vLLM, `gpu_memory_utilization = 0.55`	**12,4 GB**	**giữ chỗ cứng**, gồm cả KV cache. Không phải phần vLLM thực dùng
Qwen2.5-VL-3B **bf16**	**~7,5 GB**	3,75B × 2 byte. Bản 2B cũ là ~4,5 GB
`bge-m3` embedder	~1,2 GB	**đã chuyển sang CPU**
`ViRanker` reranker	~1,2 GB	**đã chuyển sang CPU**
Còn trên GPU	**~19,9 GB / 22,5 GB**	dư ~2,6 GB cho activation và phân mảnh

Chuyển embedder với reranker sang CPU là cách đã dùng để lấy lại 2,4 GB. Truy vấn RAG thưa, vài lượt mỗi phút chứ không phải mỗi token, nên đổi ~200ms độ trễ lấy 2,4 GB là hời.

Hệ quả cho chu kỳ này. `L4 22,5GB là sàn, T4 16GB không đủ` (PHIEN_DO_COLAB.md nêu thẳng). Và yêu cầu `ctx 32k` của S2 **chưa ai đo** trên cấu hình thật.

Tính thử để biết nên trông đợi gì, **con số dưới là ước lượng cần BR-04 xác nhận**. KV cache của Qwen3-8B tốn khoảng `2 × số_lớp × số_đầu_KV × head_dim × 2 byte` mỗi token. Với 36 lớp, 8 đầu KV, head_dim 128 thì ra ~0,147 MB mỗi token, tức **~4,6 GB cho một chuỗi dài trọn 32k**. Cộng ~6 GB weights AWQ thì một chuỗi 32k chiếm ~10,6 GB trong bể 12,4 GB.

Đánh đổi thật của chu kỳ này là **ctx so với số request chạy đồng thời**. Ở 32k, bể KV gần như chỉ đủ cho một chuỗi dài trọn vẹn, và continuous batching của vLLM chỉ phát huy khi còn chỗ cho nhiều chuỗi. BR-04 tồn tại để đo chuyện đó chứ không để khẳng định.

2.4. Hiện trạng đo được, mốc `645f4a1`

Hạng mục	Lệnh	Kết quả
Bộ test	`ENV=LOCAL pytest -q`	1003 passed

Hạng mục	Lệnh	Kết quả
Lint	`ruff check src tests offline_training`	sạch
Kiểu tĩnh	`mypy` theo danh sách CI	sạch
Secret toàn lịch sử git	`git log --all -S` 5 mẫu	sạch, mọi khớp là fixture
Nhánh chưa gộp	`git log origin/feat/lop-provider-cho-benchmark ^HEAD`	rỗng, đã gộp hết

2.5. Ràng buộc bất biến

R1–R5 ở AGENTS.md áp cho mọi yêu cầu, không ngoại lệ. Bản rút gọn: LLM không tính toán; dữ liệu khách không tới LLM bên thứ ba; hạ tầng ≤ 5.000.000đ/tháng; một khái niệm một nguồn; không hardcode phần cứng.

---

3. Kiến trúc đích của chu kỳ

3.1. Hiện trạng đường sinh text

```
chat.py ─► manager ─► ModelEngine.generate_chat
                            │
                            ├─ tokenizer = self.llm.get_tokenizer()
                            ├─ apply_chat_template(enable_thinking=False)
                            └─ run_in_executor ─► vllm.LLM.generate   ◄── TRONG TIẾN TRÌNH
```

`vllm.LLM` là **API chạy lô ngoại tuyến**. Mỗi lệnh `.generate()` chạy trọn một vòng lập lịch cho đúng lô đưa vào. Đưa vào một prompt nghĩa là lô kích thước một, nên **continuous batching bị vô hiệu hoá hoàn toàn**. Thêm vào đó, `run_in_executor` cho nhiều luồng cùng gọi một đối tượng không thread-safe. Đó là nguyên nhân hiện tượng "quá tải, treo" mà ARCHITECTURE.md §5.4 mô tả.

3.2. Đích

```
chat.py ─► manager ─► ModelEngine.generate_chat
                            │
                            └─ VLLMServerClient ──HTTP──► vllm serve Qwen3-8B
                                                          (tiến trình riêng, cổng 8001)
```

`src/core/serving.py` đã có sẵn `VLLMServerClient`, `ConcurrencyGuard`, `CompletionRequest` mang được `guided_json`, và 395 dòng test. **Chưa caller nào gọi nó.** `engine.py` import từ `serving` đúng một thứ: `guard_from_env`.

3.3. Ba cái bẫy đã xác định trong đường server

Đọc mã ngày 24/10. Cả ba đều thuộc loại hỏng im lặng, nên mỗi cái có một yêu cầu riêng kèm phép thử riêng.

#	Bẫy	Đường trong tiến trình xử ra sao	Đường server
1	**Chat template**	`self.llm.get_tokenizer().apply_chat_template(...)` tại `engine.py:354`	Client chỉ có `complete()` gọi `/v1/completions` với **prompt thô**. Không có hàm chat
2	**`enable_thinking=False`**	đặt tại `engine.py:358`	Chưa có. Qwen3 mặc định **bật** thinking, nên model xổ `<think>` trước mọi câu, trong khi grammar ép `{` là token đầu và dataset v3 train **không** think
3	**`repetition_penalty` khi bật grammar**	hạ 1,25 → 1,0 tại `engine.py:341`	`CompletionRequest` mặc định 1,25 và không chỗ nào hạ. JSON **bắt buộc** lặp token (`"typeVersion"`, `"position"`, dấu ngoặc), nên phạt lặp sẽ cấm chính các token grammar đòi

Bẫy 2 và 3 không làm chương trình đổ. Chúng làm chất lượng sinh tụt, và trong lịch sử dự án đúng loại này đã **đốt trọn một vòng đo**: phiên 15/08 sửa `additionalProperties: False` mà kết quả không đổi một đơn vị nào, vì backend giải mã bỏ qua ràng buộc đó.

---

4. Yêu cầu chức năng

BR-01. `/tools/vat` phải từ chối trường lạ thay vì tính bằng 0

Ưu tiên P0 · **Người làm** Brain-1 · **Hạn** Thứ Ba 27/10 · **Phụ thuộc** không

Vì sao. `VatRequest.items` khai là `list[dict]` (`tools.py:152`) nên pydantic không kiểm gì bên trong. Đo ngày 24/10/2026 trên chính endpoint đang chạy:

Gửi vào	`calculated_total`	Tool kết luận
`{"price": 500000, "qty": 10}`	5.400.000	hợp lệ, lệch 0
đổi `price` thành `unit_price`	**0**	**lệch 5.400.000đ, không hợp lệ**
tên trường tiếng Việt	**0**	lệch 5.400.000đ, `warnings: None`

Gõ sai một chữ thì tool phát ra lời buộc tội sổ sách chính xác tới từng đồng, và sai, không kèm cảnh báo. `vat` lại đúng là tool mà **model tự điền tham số** từ câu người dùng, vì nó không nằm trong `SYSTEM_DATA_FIELDS`. Vi phạm AGENTS §4.1 quy tắc 5.

Việc này nằm ở đây thay vì chờ, vì nó rẻ, biên rõ, có đáp án kiểm được, nên hợp với người mới nhận repo.

Yêu cầu.

1. `items` PHẢI dùng model pydantic có kiểu: `name: str`, `price: float`, `qty: float`, `is_reduced_vat: Optional[bool]`.
2. Model đó PHẢI đặt `extra="forbid"`. Trường lạ PHẢI trả **422 kèm tên trường**.
3. `price` và `qty` PHẢI bắt buộc. Thiếu PHẢI trả 422, không mặc định 0.
4. Lược đồ mới PHẢI hiện ở `GET /tools` để `agentic.py::arguments_schema` ràng buộc được model lúc sinh.

5. NÊN rà các `/tools/*` khác tìm `list[dict]` hoặc `dict` trần, ghi kết quả rà vào PR kể cả khi không sửa.

Nghiệm thu.

```
# đúng trường -> 200, lệch 0
curl -s -X POST localhost:8000/tools/vat -H "X-API-Token: $T" -H "Content-Type:
        application/json" \
  -d '{"items":
        [{"name":"X","price":500000,"qty":10,"is_reduced_vat":true}],"stated_total":5400000}'
# sai tên trường -> 422, KHÔNG phải 200 kèm lệch 5,4 triệu
curl -s -o /dev/null -w "%{http_code}\n" -X POST localhost:8000/tools/vat -H "X-API-Token: $T"
        \
  -H "Content-Type: application/json" \
  -d '{"items":[{"name":"X","unit_price":500000,"qty":10}],"stated_total":5400000}'
```

Kèm test hồi quy phủ cả ba ca trong bảng. `pytest -q` PHẢI vẫn xanh.

Ngoài yêu cầu. Không đổi công thức VAT. Không đụng `vat_catalog_audit`. Không sửa tool khác, chỉ rà và ghi.

BR-02. Thêm đường chat cho `VLLMServerClient`

Ưu tiên P0 · **Người làm** Brain-2 · **Hạn** Thứ Tư 28/10 · **Phụ thuộc** không

Vì sao. Bẫy 1, 2 và 3 ở §3.3. Client hiện chỉ có `complete()` gọi `/v1/completions` với prompt thô, trong khi `generate_chat` cần chat template, và template đó đang lấy từ tokenizer của chính đối tượng `vllm.LLM` trong tiến trình. Không có đường chat thì không nối được gì.

Yêu cầu.

1. `VLLMServerClient` PHẢI có một phương thức nhận `messages` (danh sách `{role, content}`) và gọi **`/v1/chat/completions`**, để server áp chat template.
2. Phương thức đó PHẢI truyền `chat_template_kwargs: {"enable_thinking": false}`. Thiếu nó thì Qwen3 xổ `<think>`.
3. Khi có `guided_json`, `repetition_penalty` PHẢI là **1.0**, không phải mặc định 1,25.
4. PHẢI giữ nguyên hành vi lỗi hiện có: `BackendUnavailable`, `BackendTimeout`, `BackendError` ánh xạ sang mã HTTP đúng nghĩa ở tầng trên.
5. PHẢI ghim phiên bản vLLM của server khớp với tên trường client gửi. `requirements.txt` đang ghim `vllm>=0.10.2,<0.12` vì **0.12 gỡ** **`GuidedDecodingParams`**. Server dựng ở BR-04 PHẢI nằm trong cùng khoảng đó.
6. Test PHẢI dùng server giả và **đòi server giả nhận được đúng payload**: có `chat_template_kwargs`, có `guided_json`, và `repetition_penalty == 1.0`.

Nghiệm thu. `pytest tests/test_serving.py -q` xanh, và trong đó có ít nhất ba ca mới ứng với yêu cầu 2, 3, 6. Một ca PHẢI đỏ nếu cố tình bỏ `chat_template_kwargs` ra khỏi payload.

Ngoài yêu cầu. Không đụng `engine.py` ở yêu cầu này, đó là BR-03. Không làm đường vision.

BR-03. Cho `engine.py` rẽ nhánh theo backend, và chứng minh đường HTTP thi hành

Ưu tiên P0 · **Người làm** Brain-2 cặp với Brain-1 · **Hạn** Thứ Năm 29/10 · **Phụ thuộc** BR-02

Vì sao. Đây là việc trung tâm của S2. `VLLMServerClient` đã dựng xong, có test, được khai trong `docker-compose.yml`, và **không caller nào gọi nó**. Đây là lần thứ tư dự án dính hình dạng lỗi "một lớp phòng thủ trông như đã có mà chưa nối"; ba lần trước là `API_TOKEN` so với `API_AUTH_TOKEN`, `arguments_schema()` không đóng object, và trần token benchmark rộng hơn production.

`tests/test_deploy_config.py` **không bắt được** lỗ này: nó quét regex, thấy `os.getenv("VLLM_BASE_URL")` nằm trong chính `serving.py`, rồi kết luận biến đã được mã đọc. Đọc khác dùng.

Yêu cầu.

1. `ModelEngine` PHẢI rẽ nhánh theo **backend**, không theo tên môi trường: có `VLLM_BASE_URL` thì dùng `VLLMServerClient`; không có thì nạp trong tiến trình, giữ nguyên đường cũ cho Colab; `ENV=LOCAL` thì mock.
2. Ba đường PHẢI quyết định ở **một chỗ**, không rải điều kiện khắp các phương thức.
3. `generate_chat` và `generate_text` qua đường server PHẢI cho kết quả cùng hình dạng với đường trong tiến trình. Tầng gọi không được biết mình đang đi đường nào.
4. `GET /health` PHẢI khai backend đang dùng và trạng thái sống của vLLM server.
5. PHẢI có phép thử dựng **server giả**, gọi `generate_chat`, và **đòi server giả nhận được request**. Phép thử chỉ kiểm biến môi trường được đọc là không đủ.
6. `tests/test_deploy_config.py` PHẢI cập nhật: `VLLM_BASE_URL` và `VLLM_MODEL` ra khỏi diện nghi vấn; `REDIS_URL` và `VLLM_VISION_*` ở lại `CHUA_NOI` kèm lý do.
7. Giá trị `ENV: SERVER` trong compose PHẢI được hiểu tường minh hoặc gỡ bỏ. Mã hiện chỉ rẽ ở `== "LOCAL"` (`engine.py:181`), nên mọi giá trị khác đều rơi vào nhánh nạp model lên CUDA.

Nghiệm thu.

```
ENV=LOCAL pytest tests/test_serving.py tests/test_runtime_loading.py
        tests/test_deploy_config.py -q
ENV=LOCAL pytest -q          # 1003 passed trở lên
```

Và trên Colab ở BR-04: một câu hỏi thật đi trọn vòng qua `VLLM_BASE_URL`, log của vLLM server PHẢI ghi nhận request đó.

Ngoài yêu cầu. Không nối Redis. Không nối vision qua HTTP. Không dựng Cloudflare Tunnel.

BR-04. Dựng vLLM server Qwen3-8B trên Colab L4 và đo ngân sách ngữ cảnh

Ưu tiên P0 · **Người làm** Brain-2 và Brain-1 cùng ngồi · **Hạn** Thứ Sáu 30/10 · **Phụ thuộc** BR-03

Vì sao. S2 đòi **ctx 32k**, và ARCHITECTURE.md §9.3 giải thích vì sao đó là điều kiện cần: vòng agentic nạp lại kết quả tool ở mỗi bước, nên ở `max_model_len` nhỏ thì context tràn sau khoảng ba lần gọi tool có kèm đoạn RAG. Chưa ai đo 32k trên phần cứng thật.

Hai ràng buộc đã chốt, không đưa ra thương lượng lại trong chu kỳ này:

1. **Vision ở lại.** Quyết định của chủ dự án ngày 24/10. ANSER cần vision cho một số tác vụ, nên mọi phương án lấy chỗ bằng cách tắt vision đều bị loại.
2. **Không lượng tử hoá vision xuống 4-bit.** RAG_VLM_KE_HOACH.md §B.2.4 đã bác phương án này với lý do đứng vững: việc của vision là đọc đúng con số tiền trên tờ giấy nhàu, đó là loại tác vụ mất mát lượng tử hoá gây hại nhất, và dự án **chưa có bộ ảnh thật nào** để đo xem mất bao nhiêu.

Nên đánh đổi cần đo là **ctx so với số request đồng thời**, xem phép tính ở §2.3.

Yêu cầu.

1. PHẢI dựng `vllm serve` cho `Qwen3-8B` AWQ trên Colab L4, cổng nội bộ, phiên bản nằm trong khoảng ghim ở BR-02 yêu cầu 5.
2. Brain PHẢI gọi được qua `VLLM_BASE_URL` và trả lời đúng một câu hỏi thật đi hết vòng agentic.
3. PHẢI đo **với vision đang nạp** ở mọi cấu hình. Đo lúc vision tắt rồi suy ra là tự cho mình một ngân sách không tồn tại.
4. PHẢI đo và ghi lại ít nhất **ba mức** `max_model_len`: 8192, 16384, 32768. Mỗi mức ghi: server có dựng nổi không, VRAM đỉnh theo `nvidia-smi`, và **số token KV cache vLLM báo lúc khởi động** (dòng `GPU KV cache size`).
5. PHẢI quy số token KV cache đó ra **số request 32k chạy song song được**, vì đó là con số quyết định `ANSER_TEXT_MAX_CONCURRENT`.
6. PHẢI đo `inference_latency_p95` ở mức đồng thời mà yêu cầu 5 cho ra, không phải ở một con số đặt sẵn. Ngưỡng tham chiếu ARCHITECTURE §13 là 5.000ms.
7. Nếu 32k không vừa, PHẢI nêu phương án theo thứ tự rẻ trước, và chọn có lý do ghi lại:
   - hạ `gpu_memory_utilization` rồi đo lại, vì weights AWQ chỉ ~6 GB nên phần giữ chỗ 12,4 GB có thể rộng hơn mức cần;
   - hạ `max_model_len` xuống mức vừa, kèm con số bao nhiêu lần gọi tool thì tràn;
   - đẩy vision sang tiến trình riêng qua `vllm-vision`, vốn đã khai trong compose mà chưa nối.
8. Kết quả PHẢI thành một quyết định ghi vào ARCHITECTURE.md trong cùng PR, **kèm sửa bảng §5.1 đang lỗi thời** như §2.3 nêu.

Nghiệm thu. Một bảng trong PR, mỗi dòng một mức `max_model_len`, các cột: dựng được hay không, VRAM đỉnh, `GPU KV cache size`, số request 32k song song suy ra, p95. Kèm một đoạn log vLLM server cho thấy request của Brain đi tới nơi.

Ngoài yêu cầu. Không phục vụ traffic thật từ Colab. ARCHITECTURE.md §4.4: điều khoản Colab Paid cấm web service, nên Colab chỉ dùng để đo và thử nghiệm rời rạc. Không đổi model vision trong chu kỳ này.

BR-05. Chạy lại phiên đo trên đường server, đóng việc treo số 1 và số 2

Ưu tiên P1 · **Người làm** Brain-1 · **Hạn** Thứ Tư 04/11 · **Phụ thuộc** BR-04

Vì sao. Chủ dự án chốt ngày 24/10: đo **bản fine-tune** `anser-v3-awq`, và mục đích là "*xem hổng chỗ nào để còn biết mà cải tiến*". Nên việc này là **chẩn đoán**, không phải chấm điểm. Đầu ra có giá trị của nó là một **danh sách lỗ hổng theo loại**, đủ cụ thể để quyết vòng cải tiến sau sửa cái gì.

Nó đồng thời đóng hai việc treo ở TIEN_TRINH_BRAIN.md §3. Treo 1: `render_tools()` từng quảng cáo `sales`, `expenses`, `lines*` cho model trong khi lược đồ đã gạch chúng; sửa ở `0c82c4d` nhưng **chưa có số chứng minh nó kéo agentic lên**. Treo 2: `providers.py` ghi lại chế độ ràng buộc **dự định**, còn `smoke_test_guided()` kiểm chế độ đó có **thi hành** không.

Số phiên 15/08 để biết nên soi vào đâu trước:

Nhánh	Bản fine-tune	Đọc ra gì
extraction	**65,3%**	mạnh nhất. Nhưng đo trên dữ liệu **sinh ra**, chưa phải tin nhắn thật
n8n	67,6%	
narration	**44,4%**	chỗ hổng nặng nhất. Hỏng 11 sửa 0 so với bản gốc, **trong đó 8 ca bịa số**
agentic	**15,8%**	phần lớn thất bại đến từ **lỗi lược đồ**, nay đã sửa. Con số này nhiều khả năng đổi

Tám ca bịa số ở narration là lỗi **R1**, và với sản phẩm kế toán thì đó là loại không được phép có.

Runbook đầy đủ ở PHIEN_DO_COLAB.md. Khoảng một giờ, vì dữ liệu sinh phiên 15/08 còn trên Drive tại `ANSER_AI_Logistics/generated`.

Yêu cầu.

1. PHẢI chạy `benchmark_v3.py` trên `anser-v3-awq`, qua **đường server** của BR-03. Đo đường không dùng nữa là đo một khâu không phục vụ ai.
2. PHẢI đọc ba dòng cảnh báo khung đo **trước** điểm số: `⚠ đầu ra KHÔNG đọc được thành JSON`, `⚠ CẮT CỤT`, `[chốt chặn]`. Tỷ lệ JSON hỏng từ 30% trở lên gần như luôn là mã chứ không phải model.
3. PHẢI xác nhận `smoke_test_guided()` báo ràng buộc **đang thi hành** trên đường server. Đây là chỗ bẫy 2 và 3 ở §3.3 lộ ra nếu BR-02 làm sót.
4. PHẢI dùng chung các hằng số trần token (`MAX_DECISION_TOKENS`, `MAX_REPORT_TOKENS`, `MAX_WORKFLOW_TOKENS`). Benchmark từng đo ở 1024 trong khi production cấp 700, nên mọi tỷ lệ cắt cụt đo được đều đẹp hơn thứ khách nhận.
5. **Đầu ra chính PHẢI là bảng phân loại lỗi**, không phải một con số phần trăm. Mỗi nhánh liệt kê các loại hỏng kèm **số ca** và **một ví dụ nguyên văn**. Tối thiểu phân biệt: bịa số, sai lược đồ, cắt cụt, thiếu trường bắt buộc, hiểu sai ý câu hỏi.
6. Loại **bịa số** PHẢI soi riêng, vì nó là lỗi R1. Với mỗi ca, ghi con số model bịa và con số đúng lẽ ra phải có.
7. Mỗi loại lỗi PHẢI quy về một trong ba nguyên nhân, vì cách sửa khác hẳn nhau: **dữ liệu huấn luyện**, **lược đồ hoặc prompt**, **mã khung đo**. Lịch sử dự án cho thấy ba lần liên tiếp con số xấu đến từ mã chứ không từ model.
8. Kết quả PHẢI cập nhật vào TIEN_TRINH_BRAIN.md: treo 1 và 2 đóng hay vẫn mở.
9. NÊN chạy thêm bản gốc `Qwen3-8B` rồi so theo cặp bằng `compare_runs.py`, nếu còn thời gian GPU. Việc này trả lời câu "bản fine-tune làm hỏng câu nào mà bản gốc vốn đúng", thứ mà một lần chạy đơn lẻ không nói được. Để ở mức NÊN vì nó tốn gấp đôi thời gian GPU.

Nghiệm thu. File JSON kết quả, cộng **bảng phân loại lỗi** theo yêu cầu 5 đến 7, đính vào PR. Dòng `[chốt chặn] ràng buộc JSON: ✓ CHẠY` PHẢI có mặt.

Ngoài yêu cầu. Không huấn luyện lại model trong chu kỳ này. Bảng phân loại lỗi là **đầu vào** cho quyết định huấn luyện lại, không phải quyết định đó.

BR-06. Bộ đo RAG

Ưu tiên P1 · **Người làm** Brain-1 · **Hạn** Thứ Năm 05/11 · **Phụ thuộc** không

Vì sao. `src/core/knowledge.py` có 571 dòng và 70 test đơn vị, và **không một chỉ số recall, MRR hay faithfulness nào**. TIEN_TRINH §7 xếp việc này thứ hai và ghi rõ nó không cần GPU, nên nó chạy song song với mọi thứ trên.

Bốn lỗ hổng chức năng mà ARCHITECTURE.md §13b.3 liệt kê thì **đã vá hết**: `workspace_id` bắt buộc ở mọi lối vào, `Passage` có `source` và ngày hiệu lực, tài liệu hết hiệu lực bị loại hẳn, `ingest_folder` đã ra khỏi `__init__`. Việc còn lại thuần là đo.

Yêu cầu.

1. PHẢI báo cáo ít nhất một chỉ số tầng truy hồi và một chỉ số tầng sinh. Ngưỡng khởi điểm: **Recall@20 ≥ 0,8**, MRR càng gần 1 càng tốt.
2. Bộ câu hỏi sinh từ tài liệu PHẢI có người rà lại **20% mẫu**. Câu hỏi sinh máy mà tự trả lời được không cần truy hồi thì phải loại.
3. **PHẢI KHÔNG gửi tài liệu của khách tới DeepSeek.** BAN_GIAO_VAN_TAI.md §E.3 đề nghị dùng DeepSeek sinh câu hỏi từ "hợp đồng, bảng giá cước, chính sách công nợ", và đó là **dữ liệu khách**, vi phạm R2. DeepSeek chỉ dùng cho corpus công khai là nghị định và văn bản pháp luật. Tài liệu khách thì gõ tay hoặc sinh bằng model local.
4. `grounding.py` đã là một dạng faithfulness thu hẹp về **con số**. Yêu cầu là **mở rộng nó thành chỉ số báo cáo được**, không thay bằng thứ khác.
5. NÊN thử ba cải tiến theo thứ tự của E.3: chỉnh cách chia đoạn trước (bge-m3 nhận tới 8192 token, chia theo 128 token của model cũ là phí), rồi bật tắt ViRanker để so, rồi tìm lai từ khoá cộng vector.

Nghiệm thu. Một lệnh chạy ra bảng chỉ số, số liệu đính vào PR, kèm ghi chú nguồn câu hỏi và tỷ lệ người đã rà.

Ngoài yêu cầu. Không đổi embedder hay reranker. Không nạp thêm nguồn tài liệu mới.

BR-07. Mở rộng cổng lint của CI

Ưu tiên P2 · **Người làm** Brain-1 · **Hạn** Thứ Sáu 06/11 · **Phụ thuộc** không

Vì sao. CI chỉ chạy `ruff` trên chín file viết tay, nên nó không nhìn thấy tám lỗi đã sửa ngày 22/10. Trong đó có một lỗi `F821` làm ô 4.2 của notebook Colab ném `NameError` giữa phiên đo, sau khi đã nạp xong model. Chu kỳ này có tới hai việc chạy trên Colab, nên lỗi loại đó đắt gấp đôi.

Yêu cầu. `.github/workflows/ci.yml` PHẢI chạy `ruff check src tests offline_training`. Lỗi mới PHẢI làm CI đỏ.

Nghiệm thu. Thêm tạm một biến tên `l` vào một file trong `src/`, đẩy lên nhánh phụ, CI PHẢI đỏ. Gỡ ra, CI PHẢI xanh.

Ngoài yêu cầu. Không mở rộng `mypy` trong chu kỳ này, diện lỗi của nó rộng hơn nhiều.

5. Yêu cầu phi chức năng

Mã	Yêu cầu	Nghiệm thu
NFR-01	**Bảo mật.** Không secret trong repo, không commit `.env`. `VLLM_API_KEY` đọc từ env, không nhận qua tham số từ tầng trên	Quét secret 5 mẫu trên toàn lịch sử trước mỗi lần push
NFR-02	**Chi phí.** Chu kỳ này dùng Colab Pro đã có, **không thuê GPU**. PR nào tăng chi phí PHẢI ghi con số VNĐ/tháng	Ghi chi phí thật vào biên bản cuối chu kỳ
NFR-03	**Hiệu năng.** `inference_latency_p95` ở 4 request đồng thời PHẢI đo và ghi lại. Ngưỡng tham chiếu 5.000ms	BR-04 yêu cầu 4
NFR-04	**Quan sát được.** Mọi log PHẢI có `request_id`. `/health` PHẢI khai backend đang dùng	BR-03 yêu cầu 4
NFR-05	**R1.** Không con số tài chính nào ra từ model. Đổi đường sinh text không được nới ranh giới này	`pytest -q` xanh, gồm các ca `bat_buoc_tool`
NFR-06	**R5.** Không hardcode tên GPU, số VRAM, đường dẫn máy ai. Mọi tham số phần cứng đọc từ env	`grep -rn "content/drive" src/` rỗng
NFR-07	**Chưa biết khác bằng không.** Trường thiếu dữ liệu PHẢI là `None`, không phải `0`	Có trong test của từng yêu cầu

6. Phân công và lịch

6.1. Hai luồng việc, chia theo năng lực chứ không theo tên

Nhân sự chu kỳ này chưa chốt, nên lịch chia theo **loại việc**. Ai nhận luồng nào thì điền vào bảng lúc bắt đầu, và lịch không đổi theo.

Luồng	Tính chất	Việc
A, đường găng	Cần ngữ cảnh sẵn có về `serving.py`, `engine.py` và vòng agentic. Sai ở đây làm hỏng cả chu kỳ	BR-02 → BR-03 → BR-04 → BR-05
B, biên rõ	Phạm vi hẹp, đáp án kiểm được bằng lệnh, hỏng cũng không lan. An toàn cho người mới nhận repo	BR-01, BR-06, BR-07

Nếu chỉ có một người làm cả hai luồng: chạy luồng A theo đúng thứ tự, chèn BR-01 vào ngày đầu vì nó mất khoảng một ngày và cho người làm chạm vào lớp tất định trước khi đụng tầng model. BR-06 và BR-07 đẩy sang chu kỳ 02, không cần giải trình, vì chúng là P1 và P2.

Nếu có hai người và một người sắp nghỉ: người sắp nghỉ nhận luồng A và **dồn hết vào tuần 1**. Giá trị lớn nhất họ để lại là thứ chưa nằm trong tài liệu, nên BR-03 và BR-04 làm **theo cặp**, và người ở lại là người **viết tài liệu ngay trong lúc nghe**, không để sau. Từ ngày họ nghỉ, kế hoạch giả định không hỏi được ai nữa.

Chủ dự án: chốt §8.3, cấp Colab Pro và quyền đọc Drive `ANSER_AI_Logistics/`.

6.2. Đường găng

```
BR-02 (đường chat cho client) ─► BR-03 (nối engine) ─► BR-04 (dựng + đo trên L4) ─► BR-05 (phiên
                                                                                          │
BR-06 (bộ đo RAG) ──────── chạy song song, không cần GPU ──────────────────────────────────┘
```

Cả bốn việc đầu nằm trên một đường. **BR-02 trượt là cả chu kỳ trượt**, nên nó hạn sớm nhất và giao cho người có ngữ cảnh.

6.3. Tuần 1: Thứ Hai 26/10 đến Thứ Sáu 30/10

Ngày	Luồng A, đường găng	Luồng B, biên rõ
T2 26/10	Đọc `serving.py` và `engine.py`, chốt thiết kế rẽ nhánh backend	Dựng môi trường theo sổ tay §2, ra 1003 passed. Làm bài tập 1 và 2
T3 27/10	**BR-02**	**BR-01**
T4 28/10	**BR-02 hạn.** Bắt đầu **BR-03**	**BR-01** xong, có test hồi quy
T5 29/10	**BR-03 hạn**	Cặp trên **BR-03**, và là người **viết tài liệu**
T6 30/10	**BR-04 hạn**	Cùng ngồi **BR-04** trên Colab, tự tay chạy một lượt

6.4. Tuần 2: Thứ Hai 02/11 đến Thứ Sáu 06/11

Tuần 2 giả định người của luồng A có thể đã nghỉ, nên mọi việc dồn về người ở lại.

Ngày	Việc
T2 02/11	Đọc lại tài liệu đã viết tuần 1. Chạy lại BR-04 **một mình** để chắc là làm được mà không cần hỏi ai
T3 03/11	**BR-06** dựng bộ câu hỏi và nhãn
T4 04/11	**BR-05 hạn.** Phiên đo trên Colab. Đọc cảnh báo khung đo trước điểm số, rồi dựng bảng phân loại lỗi
T5 05/11	**BR-06 hạn.** Báo cáo chỉ số RAG
T6 06/11	**BR-07.** Viết biên bản chu kỳ, cập nhật TIEN_TRINH_BRAIN và ARCHITECTURE

6.5. Hai mốc kiểm

M1, Thứ Sáu 30/10 cuối ngày. Trả lời đúng một câu: *một câu hỏi thật có đi trọn vòng từ `/chat` qua `VLLM_BASE_URL` tới vLLM server trên L4 và quay về không, và log của server có ghi nhận request đó không?*

Không đạt thì **dừng BR-06 và BR-07**, dồn toàn bộ tuần 2 vào đường găng.

M2, Thứ Sáu 06/11. Cổng ra chu kỳ, xem §7.

---

7. Cổng ra chu kỳ

Chu kỳ 01 đạt khi đủ cả sáu:

- [ ] `/chat` chạy qua **vLLM server** chứ không phải `vllm.LLM` trong tiến trình, và có một phép thử **chứng minh server nhận được request**, không phải chỉ chứng minh biến môi trường được đọc (BR-03)
- [ ] Ba bẫy ở §3.3 đều có phép thử riêng và đều xanh: chat template, `enable_thinking=False`, `repetition_penalty=1.0` khi bật grammar (BR-02)
- [ ] Ngân sách ngữ cảnh đã **đo** trên L4 ở ít nhất ba mức `max_model_len`, **với vision đang nạp**, và quyết định ctx cùng `ANSER_TEXT_MAX_CONCURRENT` đã ghi vào ARCHITECTURE kèm lý do (BR-04)
- [ ] Bảng §5.1 của ARCHITECTURE đã sửa, không còn ghi vision AWQ 3,0 GB trong khi mã nạp bf16 7,5 GB (BR-04)
- [ ] Có **bảng phân loại lỗi** của bản fine-tune theo nhánh, mỗi loại kèm số ca và một ví dụ nguyên văn, và mỗi loại đã quy về một trong ba nguyên nhân: dữ liệu, lược đồ, hay mã khung đo (BR-05)
- [ ] `/tools/vat` trả 422 cho trường lạ, không còn tính bằng 0 rồi buộc tội sổ sách (BR-01)
- [ ] `pytest -q` xanh và `ruff` sạch, đo trên đúng mã đã chạy phiên đo
- [ ] Biên bản chu kỳ ghi: việc treo 1 và 2 đóng hay mở, chỉ số RAG nếu có, chi phí thật, và danh sách đẩy sang chu kỳ 02

Mỗi mục chưa đạt PHẢI có một dòng nêu lý do và người chịu trách nhiệm đẩy tiếp.

---

8. Rủi ro, giả định, việc cần chốt

8.1. Rủi ro

Rủi ro	Dấu hiệu sớm	Xử lý
Colab cấp T4 thay vì L4	Ô kiểm GPU ở đầu notebook	Đổi phiên, hoặc đợi. T4 16GB không đủ cho Qwen3-8B AWQ
ctx 32k không vừa cùng vision trên L4	BR-04 cấu hình thứ ba không dựng nổi	Đây là **kết quả đo**, không phải thất bại. Ghi đánh đổi vào ARCHITECTURE rồi chọn
Phiên bản vLLM trên Colab trôi khỏi khoảng ghim	`guided_json` bị bỏ qua im lặng	BR-05 yêu cầu 4 bắt được. Ghim phiên bản ngay lúc `pip install`
Brain-2 nghỉ sớm hơn dự kiến	không có dấu hiệu báo trước	Dồn BR-02 và BR-03 lên sớm. Brain-1 viết tài liệu **trong lúc** nghe, không để sau
Phiên Colab hết giờ giữa chừng	Runtime ngắt	`bat_lai_brain.py` đã có để dựng lại. Chạy bằng `%run -i`, không phải `!python`

8.2. Giả định

1. Colab Pro cấp được L4 trong các buổi đã lên lịch.
2. Bản AWQ `ANSER_AI_Logistics/anser-v3-awq` và thư mục `generated` của phiên 15/08 còn trên Drive.

3. Không thuê GPU trong chu kỳ này.
4. Hoàng Phát vẫn là khách duy nhất, nên chưa cần cô lập dữ liệu đa khách.

8.3. Bốn quyết định của chủ dự án, chốt ngày 24/10/2026

#	Câu hỏi	Quyết định	Đổi gì trong tài liệu này
1	Ai nhận luồng nào, và người sắp nghỉ đi từ ngày nào	**Chưa chốt**	§6.1 chia việc theo **loại**, không theo tên, kèm phương án khi chỉ có một người. Lịch không đổi theo nhân sự
2	Đo bản gốc hay bản fine-tune	**Bản fine-tune** `anser-v3-awq`, để xem hổng chỗ nào mà cải tiến	BR-05 đổi từ chấm điểm sang **chẩn đoán**. Đầu ra chính là bảng phân loại lỗi, không phải một con số. So cặp với bản gốc hạ xuống mức NÊN
3	Có tắt vision để lấy ctx 32k không	**Không.** ANSER cần vision cho một số tác vụ	BR-04 bỏ hẳn phương án tắt vision, đo với vision **đang nạp** ở mọi cấu hình. Đánh đổi chuyển sang ctx so với số request đồng thời
4	Khi nào tìm kế toán rà sổ	**Tính sau**, tập trung vào việc làm được ngay	Gỡ khỏi §9. Nó vẫn nằm ở GIAO_VIEC_BRAIN.md V6

8.4. Còn để ngỏ

#	Câu hỏi	Khi nào cần trả lời
1	**Ai nhận luồng A, ai nhận luồng B**	Sáng Thứ Hai 26/10, điền vào bảng §6.1. Chưa có người cho luồng B thì chạy phương án một người
2	**Nếu BR-04 cho thấy 32k không vừa, chấp nhận ctx thấp hơn hay đẩy vision sang tiến trình riêng?**	Khi có số đo, dự kiến Thứ Sáu 30/10. Hỏi trước lúc có số là đoán

---

9. Ngoài phạm vi chu kỳ 01

Nêu ra để không ai làm, và để biết chúng chưa chết.

Việc	Vì sao hoãn	Ở đâu
Nối Redis cho `TASK_REGISTRY`	Thuộc S2 nhưng TIEN_TRINH §7 xếp thứ 4 và ghi "chưa gấp". Chỉ cần khi chạy nhiều tiến trình	GIAO_VIEC V2 bước 5
Nối `vllm-vision` **qua HTTP**	TIEN_TRINH §7 thứ 5. Phụ thuộc kết quả đo ở BR-04	TIEN_TRINH §3 treo 3
Cloudflare Tunnel	Thuộc S2, nhưng phụ thuộc quyết định chỗ đặt mà chu kỳ này chưa có	ROADMAP điểm chặn 4

Việc	Vì sao hoãn	Ở đâu
Đưa lên chỗ khách truy cập được	Ô đỏ duy nhất còn lại của Giai đoạn B, và nó **chưa chết**. Lớp kế toán chạy được không cần GPU: đã kiểm `RUNTIME_PROFILE=minimal` ngày 24/10, `/tools/inventory-audit` trả 2 phát hiện với `engine_ready: false`. Hoãn vì chu kỳ này dồn vào S2, không vì nó khó	ROADMAP GĐ B
`SCHEMA MAP` khớp ERD Body + `get_report_lines`	ERD đã chốt nên nó sẵn sàng, nhưng nó không nằm trên đường găng của S2. Lấy làm việc đệm nếu BR-05 hoặc BR-06 xong sớm	GIAO_VIEC V3
Mảng vận tải	Chặn bởi khách. Hoàng Phát chưa làm vận tải	GIAO_VIEC V7
Huấn luyện lại model	Bảng phân loại lỗi của BR-05 là **đầu vào** cho quyết định này, nên phải có nó trước	ROADMAP GĐ D
Kế toán thật rà sổ	Chủ dự án chốt 24/10: tính sau, tập trung vào việc làm được ngay	GIAO_VIEC V6

---

10. Tham chiếu

Cần gì	Đọc đâu
Dựng môi trường, hiểu mã, từ điển thuật ngữ	SO_TAY_NGUOI_MOI.md
Bảy việc dài hạn kèm bối cảnh đầy đủ	GIAO_VIEC_BRAIN.md
Đang ở đâu, cái gì đã chắc, cái gì đang dở	TIEN_TRINH_BRAIN.md
Năm ràng buộc R1–R5, thứ tự ưu tiên S0–S8	AGENTS.md
Quyết định kiến trúc, ngân sách VRAM, sổ lỗi	ARCHITECTURE.md
Chạy phiên đo Colab từng ô	PHIEN_DO_COLAB.md
Cổng ra từng giai đoạn	ROADMAP.md
Ba mảng nghiên cứu sâu, gồm phương án bộ đo RAG	BAN_GIAO_VAN_TAI.md §E

Chu kỳ 02 dùng lại khuôn ở §1.2. Đầu vào của nó là biên bản §7 và danh sách §9, không phải một trang giấy trắng.